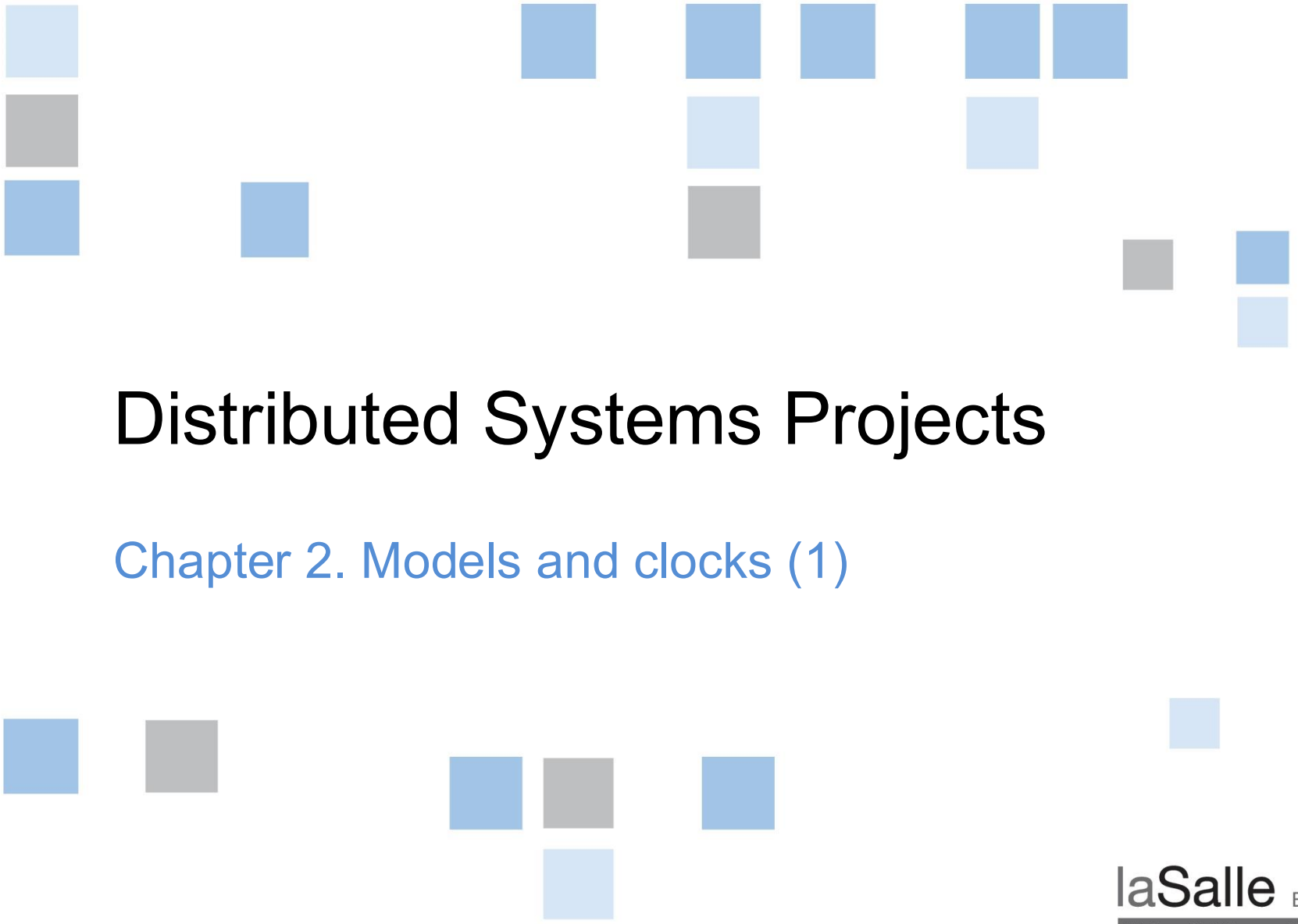


Distributed Systems Projects

Chapter 2. Models and clocks (1)

laSalle ENG
Universitat Ramon Llull

Today's agenda:

- Network Time Protocol (NTP)
- Distributed system events
- Happens-before (Causality) model
- Logical clocks
  - Lamport clock

Network Time Protocol (NTP)

The Network Time Protocol (NTP) is a protocol used to synchronize the clocks of computers and devices across a network to a precise reference time.

One the oldest internet protocols that are still in use. Designed by David L. Mills. It was first conceived in 1985 NTPv0 - RFC 958 → 2010 NTPv4 - RFC 5905.

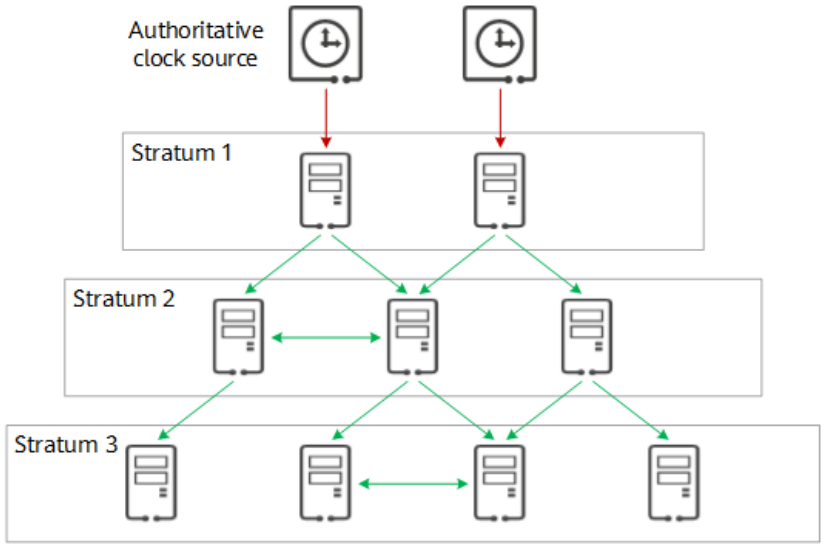
Ensures that systems have **consistent and accurate time**, which is critical for event ordering, logging, and system coordination.

Network Time Protocol (NTP)

NTP uses a hierarchical system of time sources, organized into levels known as **stratum** – 16 levels in total:

- **Stratum 0:** Reference clocks (e.g., atomic clocks, GPS clocks).
- **Stratum 1:** NTP servers directly connected to Stratum 0 devices.
- **Stratum 2 and below:** Devices synchronized from Stratum 1 or other higher stratum servers.

Client-Server Model: NTP clients request the current time from NTP servers, which respond with a timestamp to adjust the client's clock.



Network Time Protocol (NTP)

NTP is levelled as a **hierarchy** to handle scale. Millions of devices connect to NTP servers to adjust time. It is not feasible to let all of them connect to the same NTP source.

Any device, would query multiple time servers at different levels and chooses the best on to adjust time.

NTP can synchronize time to within **milliseconds** over the internet and **microseconds** on local networks, providing high accuracy.

Most big corporations manage their own NTP time servers:

- Cloudflare
- Meta
- Google

- Cool project to contribute to https://www.ntppool.org/en/

Why do we care about time?

Cool...

But Distributed Systems do not have a **shared global clock**.

It is not possible to compare timestamps across machines...

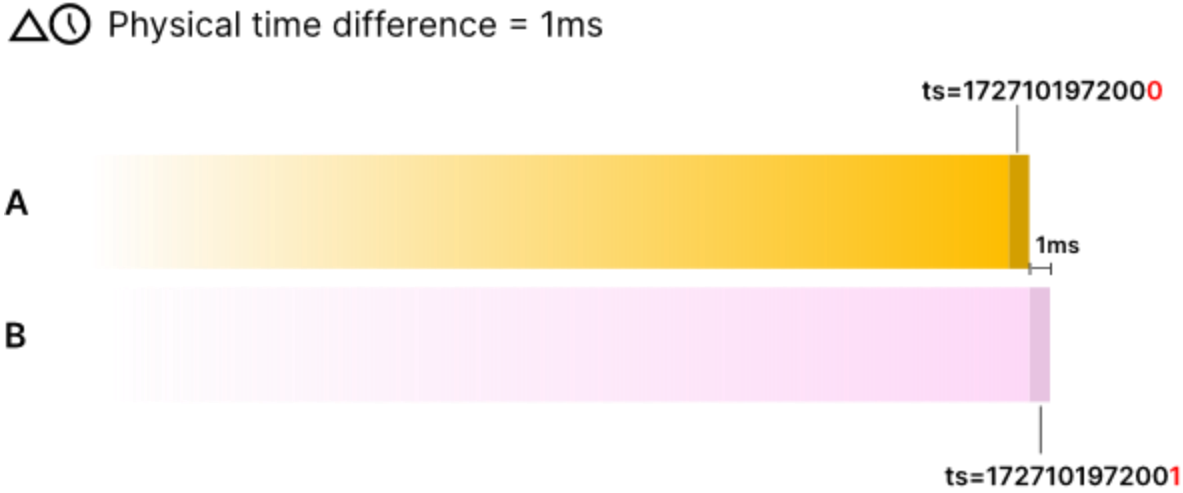
Why do we care about time?

Suppose two nodes with a physical time diff of **1ms**, and run two transactions **concurrently**.

- The first node runs a transaction X.
- The second node runs a transaction Y.

The events are logged with following data:

- **X** happened at ts = 1727101972000
- **Y** happened at ts = 1727101972001 (Event X ts + 1)



Why do we care about time?

These appear to be sequential, even though they are **concurrent**

In this example, we lose the notion of currency.

Similarly, if two transaction are not supposed to be concurrent, they may appear concurrent because of timestamp difference.

This could lead to problem and headaches when tracing bugs and troubleshooting systems.



Let’s talk about time and its importance

Wall clocks (real time)

```
// Less resolution
t_start = new Date()

cpu_goes_brrr(...)

t_end = new Date()
```

```
// More resolution
t_start = performance.now()

cpu_goes_brrr(...)

t_end = performance.now()
```

vs

Monotonic clocks

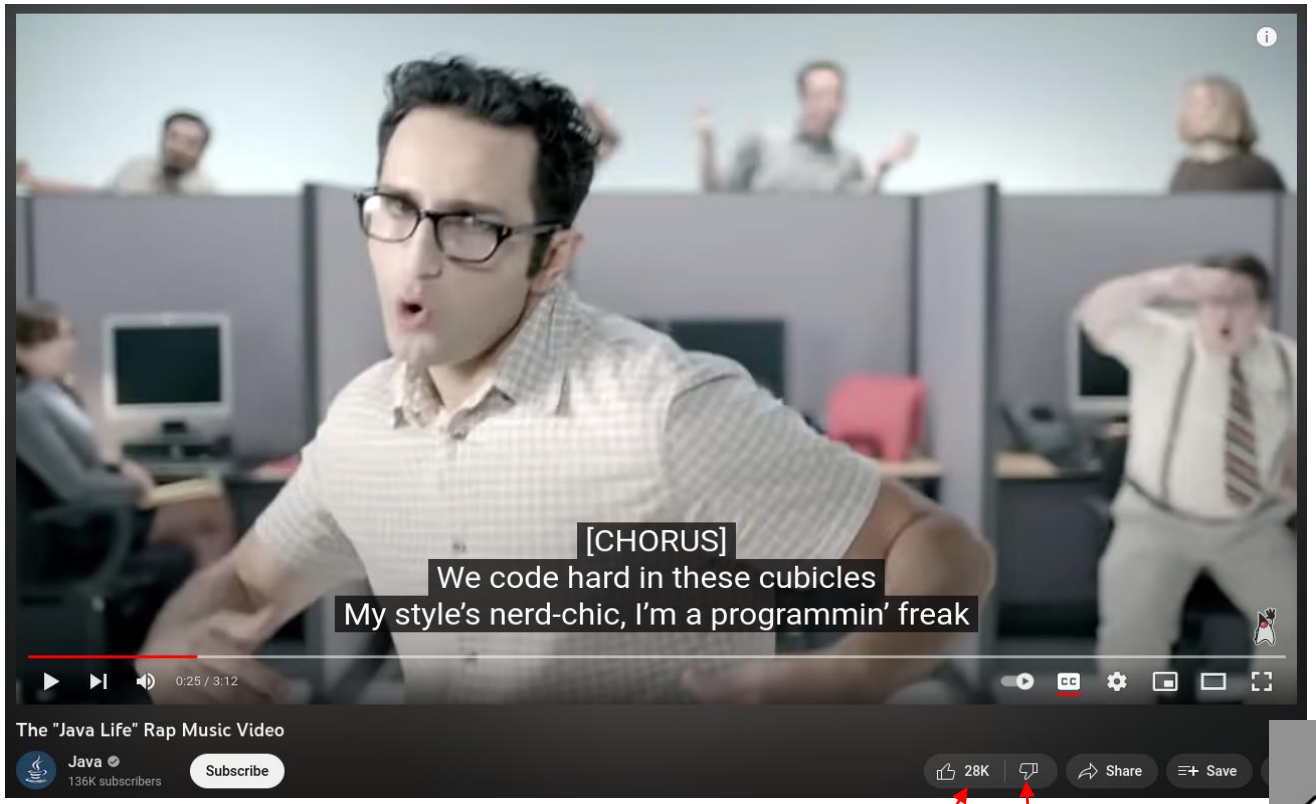
```
// Monotonically increasing

1 2 3 4 5 6 7 8 9
```

Monotonicity:

In computer science, monotonicity means that something is consistently increasing or decreasing - without reversing.

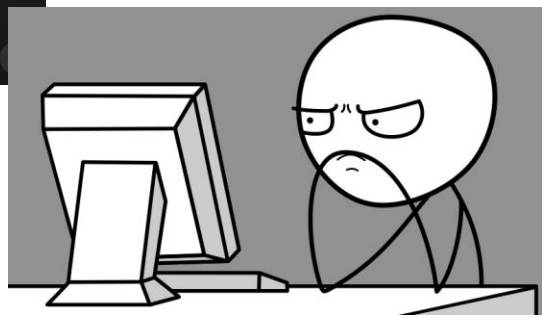
The need of events ordering





2

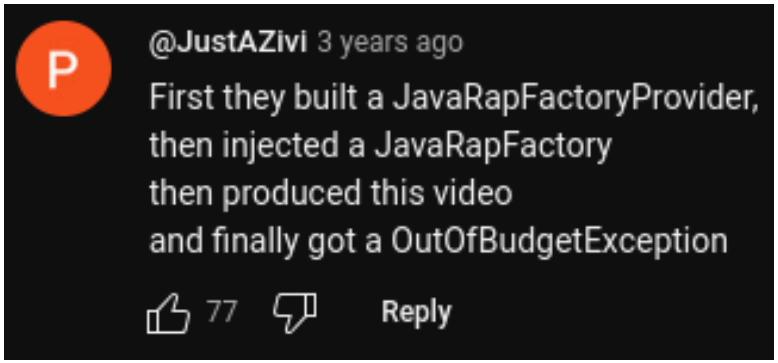
1



The need of events ordering

By the way, "Java Life" is an official Oracle song and the following comment in the YouTube section is hilarious.

Please check the video after the lecture.



Distributed system events

We need to characterize our distributed program:

1. **Local** events: Events that happen in a node and changes its state.
2. **Sending** events: A certain node sends a message to another node to inform about a change.
3. **Reception** events: A node receives a message from another one about a change.

It does not completely characterize its behavior. We need an **ordering relation!**

Happens-before model

The relation is **transitive**

$$\forall a, b, c,\ if\ a \rightarrow b,\ and\ b \rightarrow c, then\ a \rightarrow c$$

The relation is **irreflexive –** an event cannot precede itself

$$\forall a,\ a \not\rightarrow a$$

The relation is **asymmetric**

$$\forall a, b,\ if\ a \rightarrow b\ then\ b \not\rightarrow a$$

Happens-before model

Two distinct events a and b are **concurrent** if:

$$a \not\rightarrow b \; and \; b \not\rightarrow a$$

Logical clocks

The purpose of a clock is only to give us **a notion of order between events** (do not confuse with any other property associated with clocks such as elapsed time between two events).

Recall that we only have two kinds of information to determine order:

1. The order of events on a single process
2. The order between the sending and reception events of a message

Formal definition of a logical clock: *A logical clock must satisfy the following constraint:*

$$\forall a, b \in E : a \rightarrow b \implies t(a) < t(b)$$

Logical clocks: Steps of Lamport clock

1. Before event execution, a process increments its own logical clock *t(x)*
2. The **sender process** sets the message's timestamp to the new value.
3. When message is received by the receiver, the receiver sets its counter to **max(sender clock, local clock) + (one tick)**.

```java
public class LamportClock {
    private int tics;

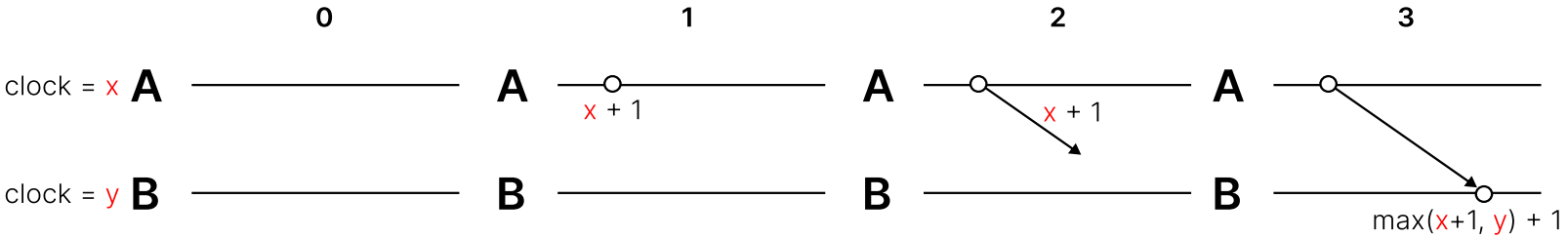
    public LamportClock() {
        this.tics = 0;
    }

    public int getValue() {
        return tics;
    }

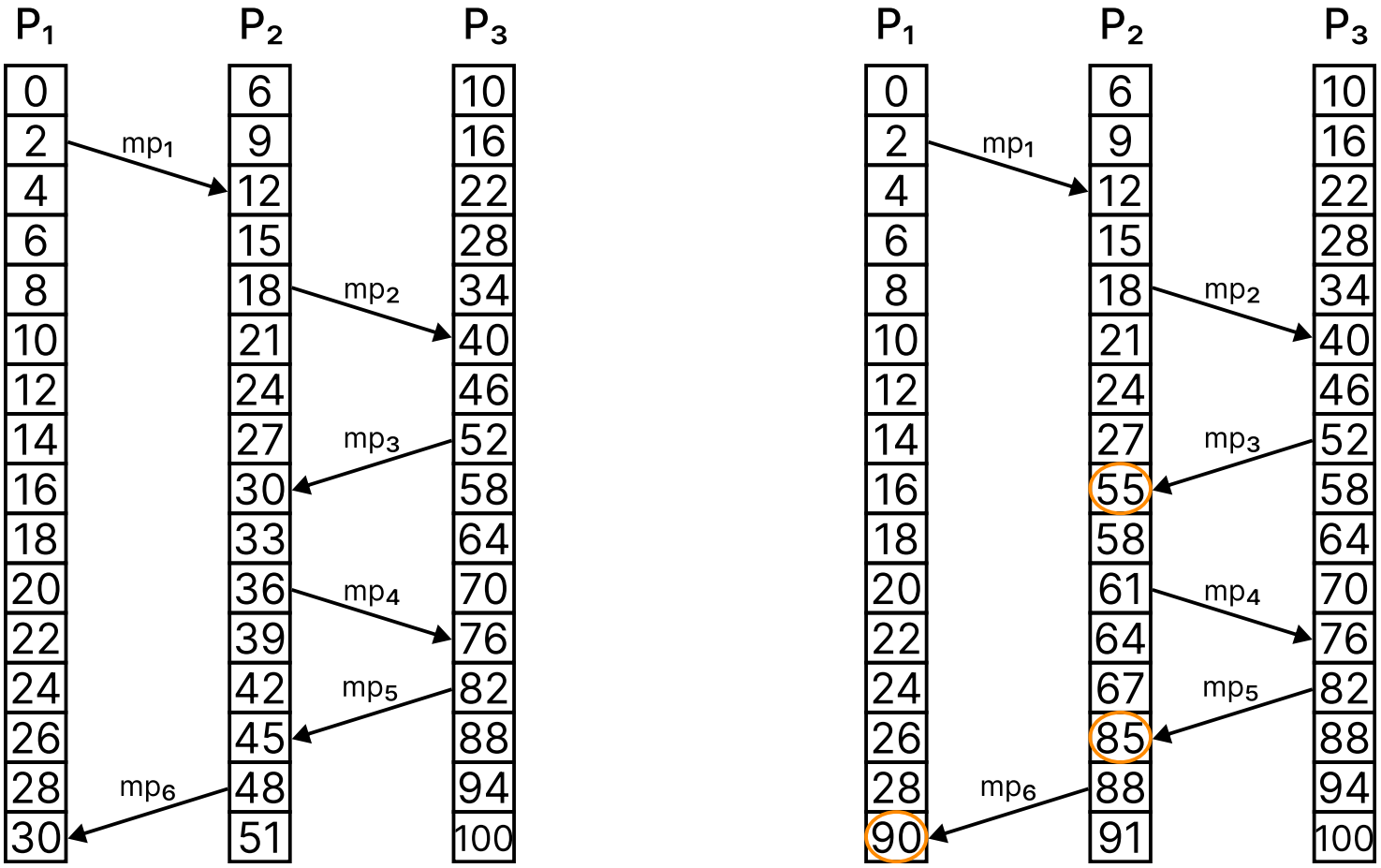
    // To be used in internal events
    public void tick() {
        tics++;
    }

    // incremented before sending
    public void sendAction() {
        tick();
    }

    public void receiveAction(int receivedValue) {
        this.ticks = Util.max(this.tics, receivedValue) + 1;
    }
}
```



Logical clocks: Lamport clock



Circled slots are when clocks are adjusted to attain **happens-before** relation

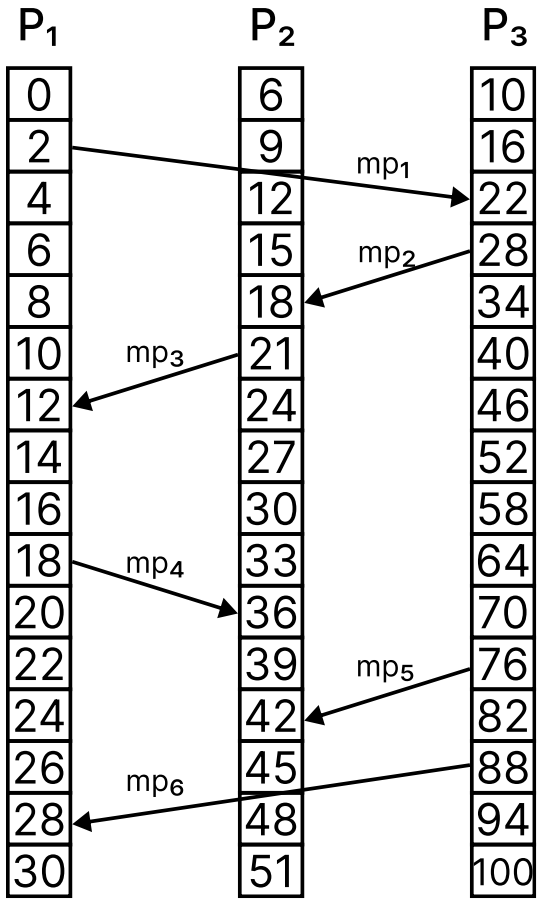
Logical clocks: Lamport clock exercise 1

Cluster of three nodes {**P**$_1$, **P**$_2$, **P**$_3$}

P1 monotonic clock increases @ **2** units

P2 monotonic clock increases @ **3** units

P3 monotonic clock increases @ **6** units



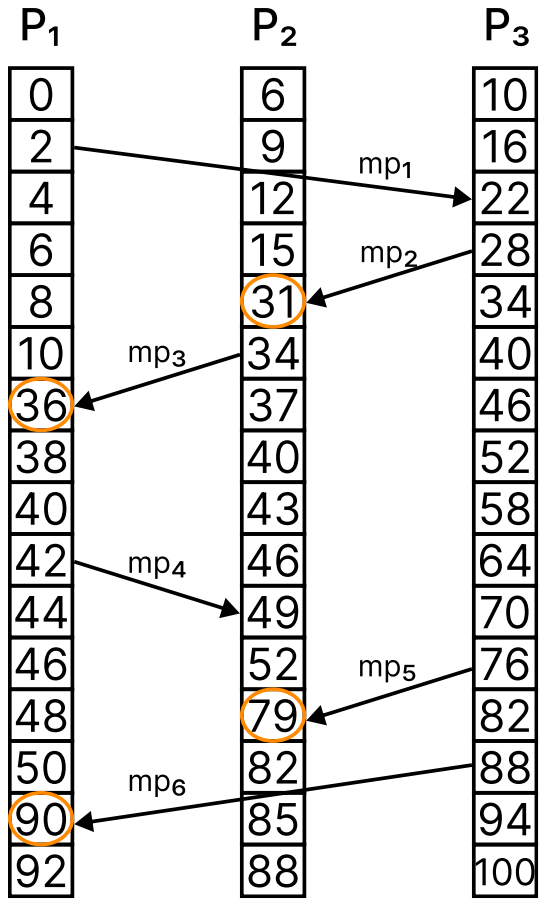
Logical clocks: Lamport clock solution 1

Cluster of three nodes {**P**$_1$, **P**$_2$, **P**$_3$}

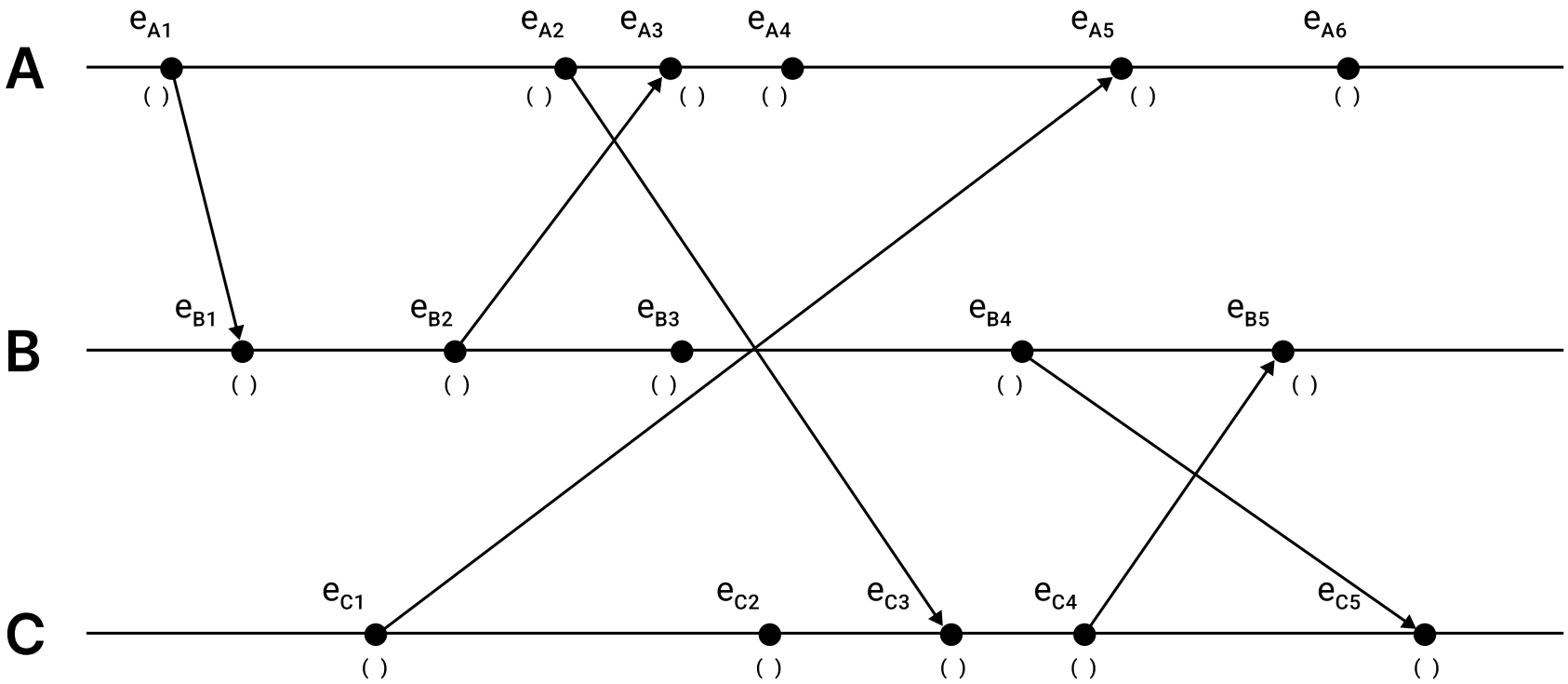
P1 monotonic clock increases @ **2** units

P2 monotonic clock increases @ **3** units

P3 monotonic clock increases @ **6** units

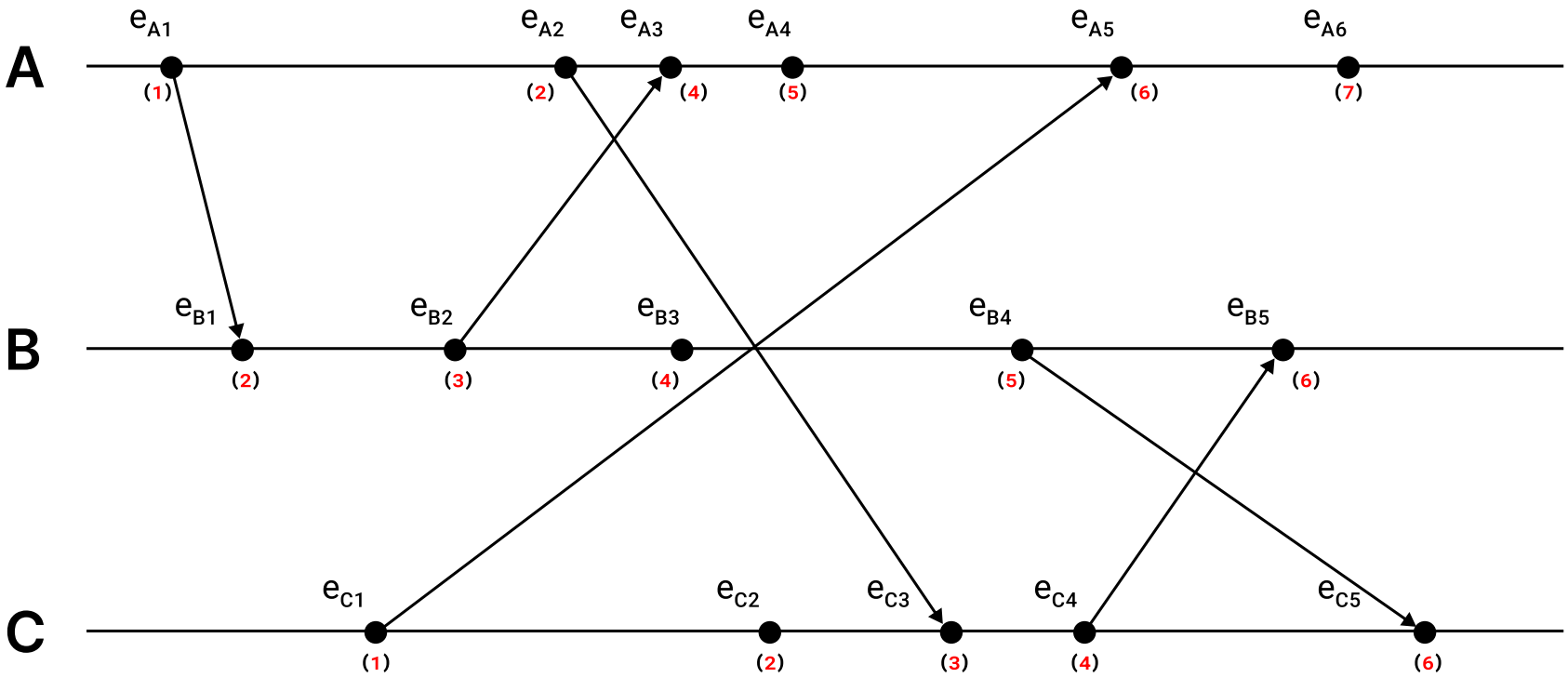


Logical clocks: Lamport clock exercise 2



All clocks start at ts=0

Logical clocks: Lamport clock solution 2



All clocks start at ts=0

Recommended references

Network Time Protocol Rev3 RFC 1305
https://www.rfc-editor.org/rfc/rfc1305.pdf

Time, clocks, and the ordering of events in a Distributed System
https://lamport.azurewebsites.net/pubs/time-clocks.pdf